

# Memento

---

# Что это такое?

«Снимок» или «Хранитель» – это поведенческий шаблон проектирования, позволяющий зафиксировать и сохранить внутреннее состояние объекта так, чтобы позднее восстановить его в это состояние

## Memento (Снимок)

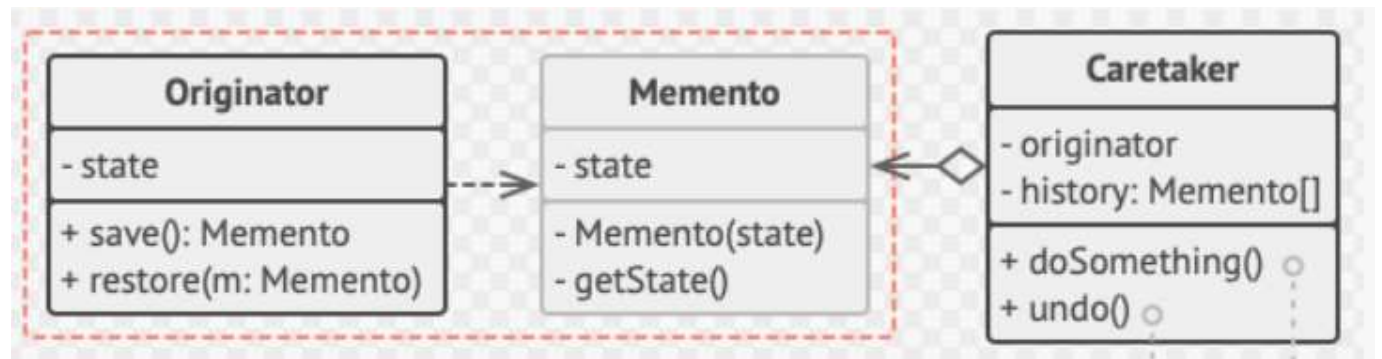


# Структура

Создатель (Originator) – создаёт снимки своего состояния.

Снимок/Хранитель (Memento) – содержит состояние создателя.

Опекун (Caretaker) – хранит историю состояний, и когда нужно, восстанавливает



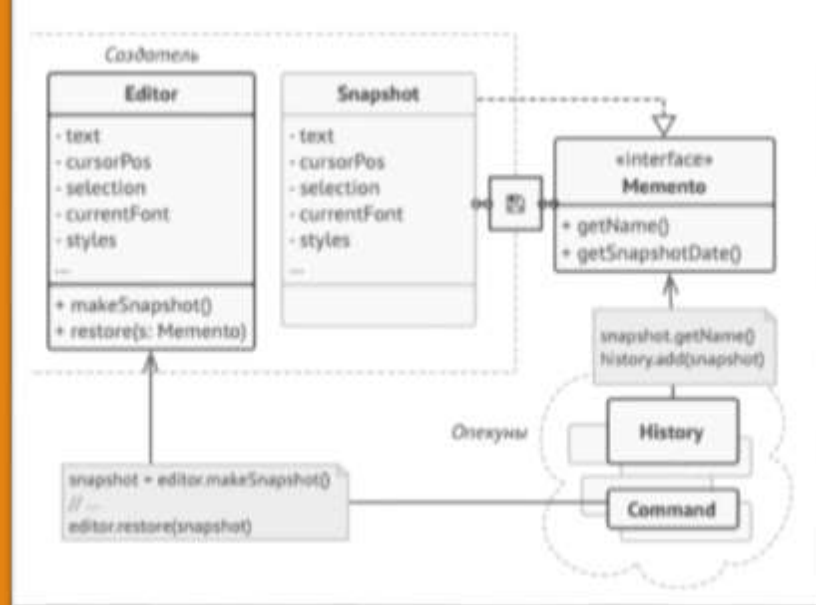
# Варианты реализации

**Классический** – даёт доступ к состояниям «Создателя» и «Снимка» через методы.

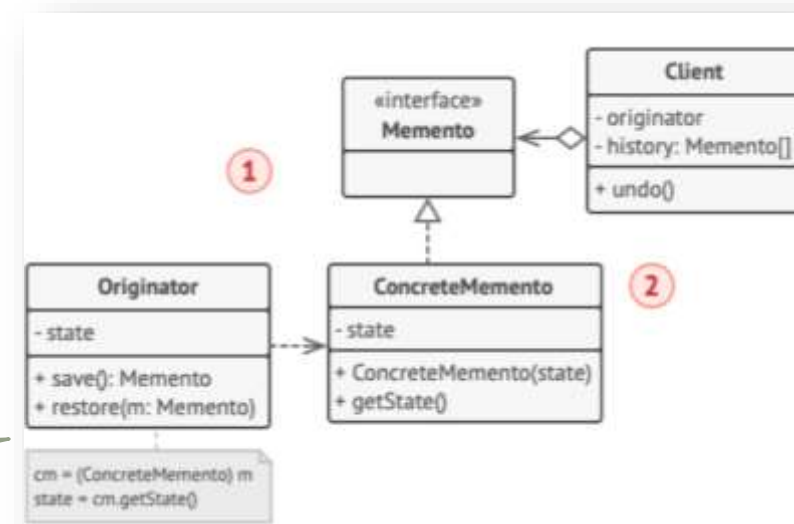
**Нестандартный** – обеспечивает прямой доступ к состояниям.

**Вариант с пустым промежуточным интерфейсом** – подходит для языков, не имеющих механизма вложенных классов.

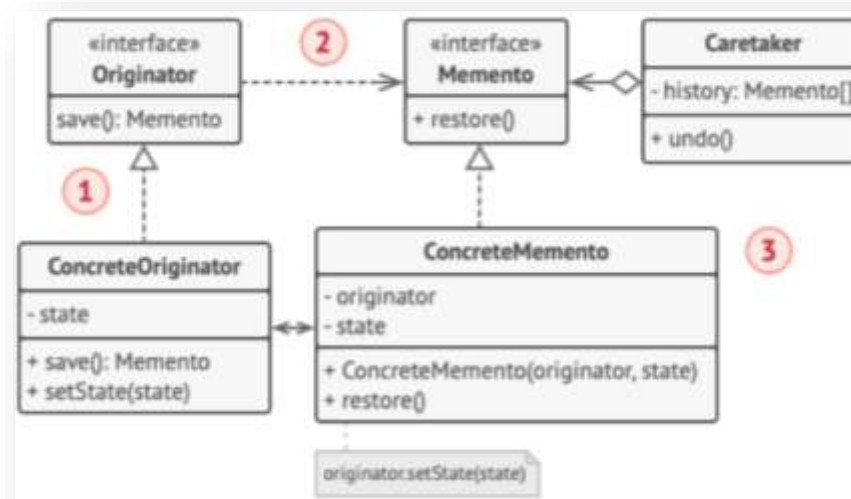
**Вариант снимка с повышенной защитой** – полностью исключает доступ к состоянию «Создателей» и «Снимков».



Классический метод



Метод с промежуточным интерфейсом



Более защищённый метод

# Реализация кода «С повышенной защитой»

```
1  from abc import abstractmethod, ABC
2
3
4  class Vacancy(ABC):
5      @abstractmethod
6      def get_vacancy_count(self):
7          """ Получить кол-во вакансий. """
8          pass
9
10     @abstractmethod
11     def set_vacancy_count(self):
12         """ Установить новое кол-во вакансий. """
13         pass
14
15     @abstractmethod
16     def save(self):
17         """ Сохранить состояние. """
18         pass
19
20     @abstractmethod
21     def restore(self, memento):
22         """ Восстановить состояние. """
23         pass
24
```

```
25
26  class MementoVacancy(ABC):
27      """ Класс, где мы можем получить состояние для текущей итерации объекта. """
28
29      @abstractmethod
30      def get_state(self):
31          """ Получить состояние. """
32          pass
33
```

```
35  class VaultJobs(ABC):
36      """
37      Класс, который будет содержать список
38      сохранённых состояний (Что-то типо стека).
39      """
40
41      @abstractmethod
42      def save_state(self):
43          """ Сохранить состояние. """
44          pass
45
46      @abstractmethod
47      def get_previous_state(self):
48          """ Получить предыдущее состояние. """
49          pass
50
```

# Код

```
51
52 class PythonVaultJobs(VaultJobs):
53     """ Сохраняем список состояний для Python вакансий. """
54
55     def __init__(self, _jobs: Vacancy):
56         # Список, где будут находиться наши состояния
57         self.__vault_jobs = []
58         self._jobs = _jobs
59
60     def save_state(self, __state: MementoVacancy):
61         """ Сохранить состояние. """
62         self.__vault_jobs.append(__state)
63
64     def get_previous_state(self):
65         """ Получить предыдущее состояние. """
66         memento = self.__vault_jobs.pop()
67         self._jobs.restore(memento)
68
69     def show_history(self) -> list:
70         """ Получить список всех состояний. """
71         return self.__vault_jobs
72
73
74 class MementoPythonVacancy(MementoVacancy):
75     def __init__(self, _state):
76         self.__state = _state
77
78     def get_state(self):
79         return self.__state
80
81     def __repr__(self):
82         return f'<Кол-во вакансий: {self.__state} >'
83
```

```
85 class PythonVacancy(Vacancy):
86     def __init__(self):
87         self._vacancy_count = 17000
88
89     def get_vacancy_count(self) -> int:
90         return self._vacancy_count
91
92     def set_vacancy_count(self, count: int):
93         """
94         Установить новое кол-во вакансии. Если кол-во < 0,
95         то новое значение установлено не будет.
96         """
97         if abs(count) == count:
98             self._vacancy_count = count
99
100     def save(self) -> MementoVacancy:
101         return MementoPythonVacancy(self._vacancy_count)
102
103     def restore(self, memento: MementoVacancy):
104         """ Восстановить предыдущее состояние. """
105         self._vacancy_count = memento.get_state()
106         print(f'Новое кол-во вакансий: {self._vacancy_count}')
107
```

# Код

```
108
109  feb = PythonVacancy()
110
111  feb.set_vacancy_count(15000)
112
113  feb.set_vacancy_count(16000)
114
115  python_saver = PythonVaultJobs(feb)
116  python_saver.save_state(feb.save())
117
118  feb.set_vacancy_count(13000)
119  python_saver.save_state(feb.save())
120
121  feb.set_vacancy_count(12000)
122  python_saver.save_state(feb.save())
123
124  print(python_saver.show_history())
125
126  python_saver.get_previous_state()
127  python_saver.get_previous_state()
128
129  print(feb.get_vacancy_count())
```

```
[<Кол-во вакансий: 16000 >, <Кол-во вакансий: 13000 >, <Кол-во вакансий: 12000 >]
Новое кол-во вакансий: 12000
Новое кол-во вакансий: 13000
13000
```

# Плюсы и минусы

---

## Плюсы:

- Позволяет реализовать функции отмены и восстановления состояния без необходимости раскрывать внутренние детали объекта.
- Упрощает управление состоянием, особенно в сложных системах.

## Минусы:

- Может потреблять много памяти, если хранится большое количество снимков.
- Не всегда подходит для объектов с большими объемами данных, так как может привести к увеличению нагрузки на систему.



# ИСТОЧНИКИ

---

Снимок // Refactoring Guru URL: <https://refactoringguru.cn/ru/design-patterns/memento>

Паттерн ООП «Хранитель» // Tproger URL: <https://tproger.ru/articles/pattern-oop-hranitel>

Хранитель // Википедия URL: [https://ru.wikipedia.org/wiki/Хранитель\\_\(шаблон\\_проектирования\)](https://ru.wikipedia.org/wiki/Хранитель_(шаблон_проектирования))

Pattern Memento // Хабр URL: <https://habr.com/ru/articles/689948/>